

LAM7ARA

Sistema de Gestión de Inventario y Ventas

Aplicación de escritorio para punto de venta

PRIMERA ENTREGA

Autor:	Bruno Depetris
--------	----------------

Docente Tutor:	Gonzalo Juan Castillo
----------------	-----------------------

Ciclo Lectivo:	2026
----------------	------

Resumen

Resumen Ejecutivo

Lam7ara es una aplicación de escritorio desarrollada con Windows Forms sobre .NET Framework 4.8, diseñada como solución integral de punto de venta y gestión de inventario para comercios minoristas de tecnología. El sistema permite administrar catálogos de productos, registros de clientes, transacciones de venta con múltiples métodos de pago, operaciones de caja registradora y un dashboard de control, todo conectado a una base de datos SQLite local almacenada en el directorio AppData del usuario.

Contexto del Problema y Motivación

En el ámbito de los comercios minoristas, especialmente aquellos orientados a la venta de productos tecnológicos (celulares, accesorios, etc.), existe la necesidad de contar con una herramienta que centralice las operaciones diarias: control de stock, registro de ventas, gestión de clientes y seguimiento de caja.

Muchos comercios aun operan con registros manuales o herramientas dispersas que generan ineficiencias, errores y pérdida de trazabilidad. Lam7ara surge como respuesta a esta necesidad, ofreciendo una solución de escritorio robusta, intuitiva y adaptada al mercado argentino, incluyendo integración con la cotización del dólar en tiempo real a través de la API de dolarapi.com.

Objetivos del proyecto

Objetivo general

Desarrollar una aplicación de escritorio de gestión de inventario y ventas que permita a comercios minoristas administrar de manera eficiente sus productos, clientes, transacciones comerciales y operaciones de caja, proporcionando una interfaz moderna e intuitiva con integración de datos financieros en tiempo real.

Objetivos específicos

- Implementar un modulo de gestión de inventario (FormStock) que permita registrar, modificar, eliminar y consultar productos con sus especificaciones técnicas, niveles de stock y precios mediante ProductoService.
- Desarrollar un modulo de procesamiento de ventas (FormVender) que vincule múltiples productos, clientes y métodos de pago, descuente stock automáticamente vía transacción SQLite y genere comprobantes en PDF con iTextSharp.
- Crear un modulo de gestión de clientes (FormCLIENTES) con operaciones CRUD completas mediante ClienteService, con búsqueda por nombre, apellido y DNI.

- Implementar un sistema de caja registradora (FormMovimientos) que permita abrir y cerrar turnos, registrar movimientos de efectivo y calcular saldo en tiempo real.
- Integrar un panel de control (FormInicio) que muestre indicadores clave: ventas del día, cantidad de ventas, clientes registrados, estado de caja y stock bajo.
- Integrar la API REST de dolarapi.com para mostrar cotizaciones del dolar (oficial, blue, tarjeta) actualizadas manualmente desde el dashboard.
- Generar comprobantes de venta en PDF con iTextSharp y gestionar un historial de ventas filtrable con opcion de anulacion (FormVerVentas).

Objetivos Específicos

- Implementar un módulo de gestión de inventario que permita registrar, modificar, eliminar y consultar productos con sus especificaciones técnicas (marca, modelo, almacenamiento, batería), niveles de stock y precios.
- Desarrollar un módulo de procesamiento de ventas que vincule productos, clientes y métodos de pago, con capacidad de generar comprobantes en formato PDF.
- Crear un módulo de gestión de clientes con operaciones CRUD completas y validación de datos (nombre, DNI, teléfono, correo electrónico).
- Implementar un sistema de caja registradora que permita abrir y cerrar turnos, registrar movimientos de efectivo y mantener un historial de operaciones.
- Integrar un panel de control (Dashboard) que muestre indicadores clave como la inversión total en inventario y la cotización del dólar en tiempo real.
- Aplicar patrones de diseño (Singleton, List-Detail) y operaciones asíncronas para garantizar la eficiencia, mantenibilidad y responsividad de la aplicación.

análisis del problema y requisitos

descripción del problema

Los comercios minoristas de tecnología enfrentan desafíos al gestionar su operativa diaria sin un sistema integrado: el control de stock se realiza manualmente, las ventas no quedan registradas de manera trazable, la información de clientes se dispersa en múltiples fuentes, y el arqueo de caja carece de automatización.

Esto genera errores en los pedidos, pérdidas de inventario no detectadas, falta de historial de compras por cliente y dificultades para conocer el estado financiero del negocio en tiempo real.

Alcance del Sistema

El sistema abarca la gestión completa del ciclo comercial de un negocio minorista: desde el alta de productos en el inventario, pasando por la gestión de clientes, el registro de ventas con emisión de comprobantes PDF, hasta el control de operaciones de caja. Adicionalmente incorpora un dashboard con datos analíticos e integración con API externa para cotización de divisas.

Limitaciones

- El sistema opera exclusivamente en entornos Windows (.NET Framework 4.8 / Windows Forms).
- La base de datos SQLite se almacena localmente en %AppData%\Lam7ara\; no soporta operación multi-equipo ni sincronización entre puntos de venta.
- No cuenta con sistema de autenticación de usuarios ni roles de seguridad implementados.
- La actualización de cotizaciones del dólar es manual (botón Actualizar); no se refresca automáticamente.
- El modulo Tecnico del sidebar no se encuentra implementado en la version actual.

Requerimientos Funcionales

ID	Requerimiento	Descripción
RF 01	Gestion de Productos	CRUD completo de productos con especificaciones técnicas (marca, modelo, almacenamiento, batería), stock y precios.
RF 02	Gestión de Clientes	CRUD de clientes con validación de datos: nombre, DNI, teléfono, email.
RF 03	Procesamiento de Ventas	Registro de ventas vinculando multiples productos, clientes y metodos de pago. Descuento automatico de stock al concretar.
RF 04	Generación de PDF	Emisión de comprobantes de venta en formato PDF mediante iTextSharp.
RF 05	Caja Registradora	Apertura/cierre de turnos y registro de movimientos de efectivo.
RF 06	Dashboard	Visualizacion de ventas del dia, cantidad de ventas, clientes registrados, estado de caja, stock bajo y ultimas ventas.
RF 07	Cotización de Divisas	Consulta a la API de dolarapi.com para obtener tipos de cambio (oficial, blue, tarjeta) actualizados.
RF 08	Navegación Centralizada	Interfaz principal (FormPrincipal) con navegación a todos los módulos del Sistema.

Requerimientos No Funcionales

ID	Requerimiento	Descripcion
RNF 01	Seguridad de Datos	Consultas parametrizadas (SQLiteCommand con Parameters) para prevenir inyección SQL.
RNF 02	Usabilidad	Interfaz dark theme moderna con controles ReaLTaiizor. Sidebar con indicador de ítem activo y reloj en tiempo real.
RNF 03	Mantenibilidad	Arquitectura en capas con separación entre modelos (Models), servicios (Services) y formularios (Forms).
RNF 04	Rendimiento	Conexion SQLite local via Conectar.ObtenerConexion() con cierre explicito usando using en cada operacion.
RNF 05	Portabilidad de Datos	La BD se almacena en %AppData%\Lam7ara\ con copia de respaldo automatica en %AppData%\BackUpLam7ara\.
RNF 06	Integridad Transaccional	Las operaciones de venta usan SQLiteTransaction para garantizar atomicidad (descuento de stock + registro de venta).

Diseño del software

Arquitectura General

El sistema implementa una arquitectura de tres capas que separa claramente presentación, lógica de servicios y acceso a datos:

Capa de presentación (Forms) - Formularios Windows Forms organizados por modulo:

- FormMAIN (navegacion con sidebar)
- FormInicio (dashboard)
- FormVender (ventas),
- FormVerVentas (historial)
- FormCLIENTES (clientes)
- FormStock (inventario)
- FormMovimientos (caja)

Los formularios hijo se cargan dinámicamente en panelMainContent del FormMAIN.

Capa de Servicios (Services) - Clases de servicio independientes que encapsulan la lógica de negocio:

- ClienteService,
- ProductoService,
- VentaService,
- CajaService
- MovimientoService.

Cada servicio expone métodos como Listar(), BuscarPorID(), Buscar(), Agregar(), Editar() y Eliminar(). VentaService.Agregar() ejecuta todo en una SQLiteTransaction para garantizar atomicidad.

Capa de Datos (Models + Conectar) - Modelos POCO que mapean las tablas SQLite:

- Cliente,
- Producto,
- Venta,
- VentaProducto,
- Caja, Movimiento.

La clase estatica Conectar gestiona la ubicacion y acceso a la base de datos Lam7araDataBaseAPP.db, con metodos Comprobar() (crea la BD en AppData si no existe), CrearBackUp() y ObtenerConexion().

Capa de APIs externas:

- La clase ApiCotizacion (HTTP client) consulta dolarapi.com y el modelo ModeloApiCotizacion deserializa la respuesta JSON via Newtonsoft.Json.

Tecnologías a Utilizar

Componente	Tecnología	Versión
Backend / Runtime	.NET Framework (C#)	4.8
Frontend / UI	Windows Forms + RealTailor	3.8.1.1
Base de Datos	MySQL (MySql.Data)	1.0.119
Generación de PDF	iTextSharp	5.5.13.4
Serialización JSON	Newtonsoft.Json	13.0.3
Configuración	System.Configuration.ConfigurationManager	9.0.1
API Externa	dolarapi.com (REST)	-
IDE	Visual Studio	2026

Diseño del software

A continuación se presenta la planificación temporal del proyecto en formato de diagrama de Gantt, dividida en las siguientes fases: análisis y relevamiento de requerimientos, creación de estructura y definición de tecnologías, desarrollo de base de datos y servicios, diseño e implementación de formularios, integración de servicios externos y publicación de la aplicación.

